

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

PRACTICAL - 1

Problem :- A student wants to travel from Home to College using the available road network. Each road connects different locations, forming a graph. The objective is to implement the Breadth First Search (BFS) and Iterative Depth First Search (DFS) algorithms to find a route from Home to College. After implementing both algorithms, compare their efficiency and performance based on the path found, time to reach, total distance and travelled.

AIM :-

- A) Implement the Breadth First Search algorithm to solve a given problem.
- B) Implement the Iterative Depth First Search algorithm to solve the same problem.
- C) Compare the performance and efficiency of both algorithms.

Code :-

1. Practical_1 Code.py

```
from collections import deque
import time
graph = {
    "Home": [("Bus Stop", 2), ("Hospital", 3), ("Park", 2)],
    "Bus Stop": [("Railway", 2)],
    "Railway": [("Library", 1)],
    "Library": [("College", 2)],
    "Hospital": [("Temple", 2)],
    "Temple": [("Garden", 2)],
    "Garden": [("College", 2)],
    "Park": [("Mall", 2)],
    "Mall": [("Cinema", 2)],
    "Cinema": [("Stadium", 2)],
    "Stadium": [("College", 3)],
    "College": []
}
def bfs(start, goal):
    queue = deque([(start, [start], 0)])
    visited = set()
    start_time = time.perf_counter()
    while queue:
        node, path, distance = queue.popleft()
        if node == goal:
            end_time = time.perf_counter()
            return path, distance, (end_time - start_time) * 1000
        if node not in visited:
            visited.add(node)
            for next_node, dist in graph[node]:
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

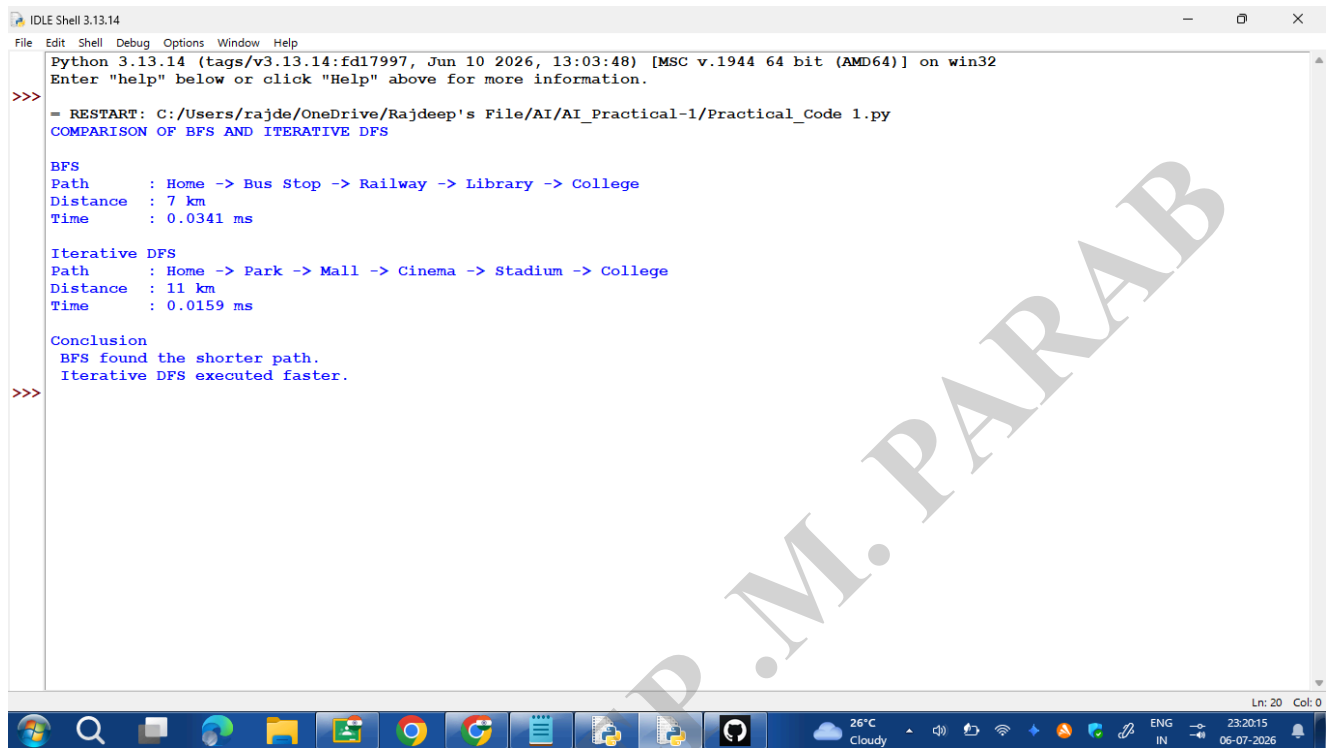
SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

```
        queue.append((next_node, path + [next_node], distance + dist))
def dfs(start, goal):
    stack = [(start, [start], 0)]
    visited = set()
    start_time = time.perf_counter()
    while stack:
        node, path, distance = stack.pop()
        if node == goal:
            end_time = time.perf_counter()
            return path, distance, (end_time - start_time) * 1000
        if node not in visited:
            visited.add(node)
            for next_node, dist in graph[node]:
                stack.append((next_node, path + [next_node], distance + dist))
bfs_path, bfs_distance, bfs_time = bfs("Home", "College")
dfs_path, dfs_distance, dfs_time = dfs("Home", "College")
print("COMPARISON OF BFS AND ITERATIVE DFS")
print("\nBFS")
print("Path      :", " -> ".join(bfs_path))
print("Distance :", bfs_distance, "km")
print("Time      :", round(bfs_time, 5), "ms")
print("\nIterative DFS")
print("Path      :", " -> ".join(dfs_path))
print("Distance :", dfs_distance, "km")
print("Time      :", round(dfs_time, 5), "ms")
print("\nConclusion")
if bfs_distance < dfs_distance:
    print(" BFS found the shorter path.")
elif bfs_distance > dfs_distance:
    print(" DFS found the shorter path.")
else:
    print(" Both algorithms found the same distance.")
if bfs_time < dfs_time:
    print(" BFS executed faster.")
else:
    print(" Iterative DFS executed faster.")
```

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

Output :-



```
IDLE Shell 3.13.14
File Edit Shell Debug Options Window Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/rajde/OneDrive/Rajdeep's File/AI/AI_Practical-1/Practical_Code 1.py
COMPARISON OF BFS AND ITERATIVE DFS

BFS
Path : Home -> Bus Stop -> Railway -> Library -> College
Distance : 7 km
Time : 0.0341 ms

Iterative DFS
Path : Home -> Park -> Mall -> Cinema -> Stadium -> College
Distance : 11 km
Time : 0.0159 ms

Conclusion
BFS found the shorter path.
Iterative DFS executed faster.
>>>
```

The screenshot shows a Python IDE window titled 'IDLE Shell 3.13.14'. The window contains the output of a script that compares Breadth-First Search (BFS) and Iterative Depth-First Search (DFS). The output shows that BFS found a shorter path (7 km) compared to Iterative DFS (11 km), and that Iterative DFS executed faster (0.0159 ms) than BFS (0.0341 ms). The script is located at 'C:/Users/rajde/OneDrive/Rajdeep's File/AI/AI_Practical-1/Practical_Code 1.py'. The window also displays the Python version (3.13.14) and the operating system (win32). The taskbar at the bottom shows various application icons and system information, including the date and time (23:20:15, 06-07-2026).